

Algorytmy Optymalizacji Dyskretnej 2025/26

LABORATORIUM 3

Porównanie algorytmów najkrótszych ścieżek

Imię i Nazwisko: Jędrzej Sajnog

Nr indeksu: 279701

9 grudnia 2025

Spis treści

1	Wstęp	3
2	Opis i pseudokody algorytmów	3
2.1	Dijkstra (Kopiec Binarny)	3
2.2	Algorytm Diala	4
2.3	Radix Heap	4
3	Wyniki eksperymentów - Wykresy	6
3.1	Long C	6
3.2	Long n	6
3.3	Rodzina Square C	7
3.4	Rodzina Square n	7
3.5	random 4n	7
3.6	Rodzina random 4C	8
4	Wykresy słupkowe dla sieci drogowej USA-road-d	8
5	Wnioski i interpretacja	9
6	Tabela wyników dla największych instancji eskperymenty P2P	10
6.1	Wnioski P2P	11

1 Wstęp

Celem niniejszego laboratorium było zaimplementowanie oraz porównanie wydajności trzech algorytmów, służącego do wyznaczania najkrótszych ścieżek w grafie z nieujemnymi wagami krawędzi. Badane algorytmy to:

- Algorytm Dijkstry (wykorzystujący kolejkę priorytetową zaimplementowaną jako kopiec binarny). $O((|V| + |E|)\log |V|)$
- Algorytm Diala (wykorzystujący kubelki $C + 1$ kubelków). $O(|E| + |V|C)$
- Algorytm Radix Heap. $O(|E| + |V|\log(VC))$

Testy przeprowadzono na rodzinach grafów dostarczonych w ramach zestawu danych 9th DIMACS Implementation Challenge (Random4, Long, Square) oraz na grafach drogowych USA. Analizowano wpływ rozmiaru grafu (n) oraz maksymalnej wagi krawędzi (C) na czas działania poszczególnych implementacji.

2 Opis i pseudokody algorytmów

Poniżej przedstawiono pseudokody zaimplementowanych algorytmów.

2.1 Dijkstra (Kopiec Binarny)

Standardowa implementacja wykorzystująca biblioteczną kolejkę priorytetową (w C++ `std::priority_queue`). Algorytm stosuje podejście leniwe (ang. *lazy deletion*) – wierzchołki nie są usuwane z kolejki przy aktualizacji odległości, lecz ignorowane przy ekstrakcji, jeśli ich pobrany priorytet jest gorszy niż aktualnie znany.

Algorytm 1 Algorytm Dijkstry (kolejka priorytetowa)

Wejście: Graf $G = (N, A)$, nieujemne wagi w , wierzchołek startowy s , opcjonalny cel g

Wyjście: Tablica najkrótszych odległości $dist$

```
1:  $dist[0..n - 1] \leftarrow \infty$ 
2:  $dist[s] \leftarrow 0$ 
3:  $PQ \leftarrow$  pusta kolejka priorytetowa (min-heap)
4:  $PQ.push(0, s)$ 
5: while  $PQ$  nie jest pusta do
6:    $(d, u) \leftarrow PQ.top()$ 
7:    $PQ.pop()$ 
8:   if  $d \neq dist[u]$  then ▷ Ignorujemy wpisy nieaktualne
9:     continue
10:  end if
11:  if  $g \neq -1$  and  $u = g$  then
12:    break
13:  end if
14:  for każdego sąsiada  $v$  wierzchołka  $u$  o wadze  $w_{uv}$  do
15:    if  $dist[u] + w_{uv} < dist[v]$  then
16:       $dist[v] \leftarrow dist[u] + w_{uv}$ 
17:       $PQ.push(dist[v], v)$ 
18:    end if
19:  end for
20: end while
21: return  $dist$ 
```

2.2 Algorytm Diala

Implementacja wykorzystująca tablicę kubełków o rozmiarze $C+1$, gdzie C to maksymalna waga krawędzi. Obsługa kubełków realizowana jest w sposób cykliczny (modulo $C+1$). Ze względu na zastosowanie struktur (np. `unordered_set`) wewnątrz kubełków, zaimplementowano mechanizm sprawdzania, czy wyjęty wierzchołek rzeczywiście należy do aktualnie przetwarzanej odległości.

Algorytm 2 Algorytm Diala (metoda kubełkowa)

Wejście: Graf G , wierzchołek źródłowy s , maksymalna waga krawędzi C

Wyjście: Tablica odległości $dist$

```
1:  $bucketCount \leftarrow C + 1$ 
2:  $dist[0..n - 1] \leftarrow \infty$ 
3:  $dist[s] \leftarrow 0$ 
4: Utwórz tablicę kubełków  $Buckets[0..bucketCount - 1]$  (każdy jest pustym zbiorem)
5:  $Buckets[0].insert(s)$ 
6:  $currentBucket \leftarrow 0$ 
7:  $d \leftarrow 0$  ▷ bieżąca odległość logiczna
8:  $left \leftarrow 1$  ▷ liczba wierzchołków w kubełkach
9: while  $left > 0$  do
10:   while  $Buckets[currentBucket]$  jest pusty do
11:      $currentBucket \leftarrow (currentBucket + 1) \bmod bucketCount$ 
12:      $d \leftarrow d + 1$ 
13:   end while
14:   Pobierz i usuń dowolny wierzchołek  $u$  z  $Buckets[currentBucket]$ 
15:    $left \leftarrow left - 1$ 
16:   if  $dist[u] > d$  then ▷ wierzchołek trafił do zbyt wczesnego kubełka
17:      $idx \leftarrow (currentBucket + (dist[u] - d)) \bmod bucketCount$ 
18:      $Buckets[idx].insert(u)$ 
19:      $left \leftarrow left + 1$ 
20:     continue
21:   end if
22:   for każdego sąsiada  $v$  wierzchołka  $u$  o wadze  $w_{uv}$  do
23:     if  $dist[u] + w_{uv} < dist[v]$  then
24:       if  $dist[v] \neq \infty$  then ▷ usuń  $v$  ze starego kubełka
25:         Usuń  $v$  z jego aktualnego kubełka
26:          $left \leftarrow left - 1$ 
27:       end if
28:        $dist[v] \leftarrow dist[u] + w_{uv}$ 
29:        $idx \leftarrow (currentBucket + (dist[v] - d)) \bmod bucketCount$ 
30:        $Buckets[idx].insert(v)$ 
31:        $left \leftarrow left + 1$ 
32:     end if
33:   end for
34: end while
35: return  $dist$ 
```

2.3 Radix Heap

Implementacja wykorzystująca strukturę Radix Heap.

Algorytm 3 Algorytm Radix Heap

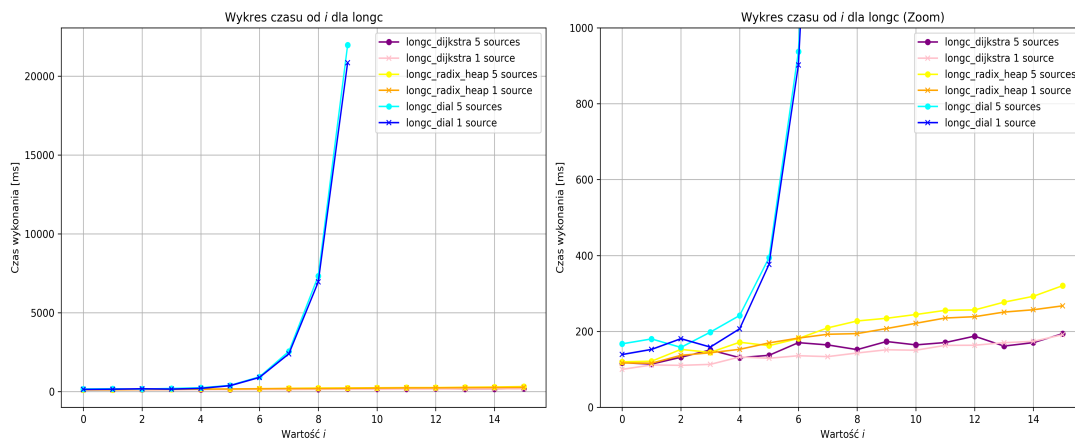
Wejście: Graf G , źródło s

```
1:  $dist[0..n-1] \leftarrow \infty$ 
2:  $dist[s] \leftarrow 0$ 
3:  $RH \leftarrow$  nowa struktura RadixHeap
4:  $RH.push(0, s)$ 
5: while  $RH$  nie jest pusty do
6:    $(d, u) \leftarrow RH.pop\_min()$ 
7:   if  $d \neq dist[u]$  then continue
8:   end if
9:   for każdego sąsiada  $v$  wierzchołka  $u$  o wadze  $w_{uv}$  do
10:     $new\_dist \leftarrow dist[u] + w_{uv}$ 
11:    if  $new\_dist < dist[v]$  then
12:       $dist[v] \leftarrow new\_dist$ 
13:       $RH.push(new\_dist, v)$ 
14:    end if
15:   end for
16: end while
```

3 Wyniki eksperymentów - Wykresy

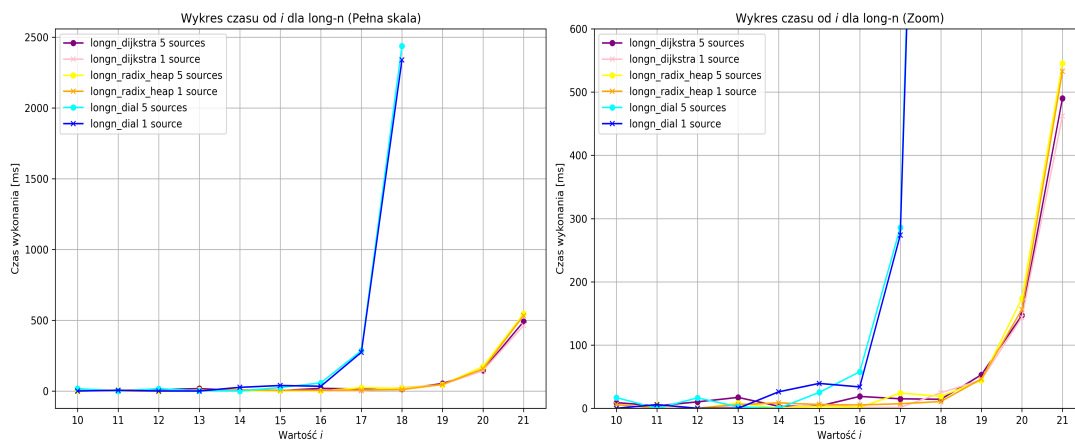
W tej sekcji przedstawiono wykresy zależności czasu działania algorytmów od parametrów wejściowych dla różnych rodzin grafów. Dla każdej rodziny liczymy najkrótszą ścieżkę od wierzchołka z najmniejszym indeksem, oraz średnią dla pięciu losowo wybranych wierzchołków.

3.1 Long C



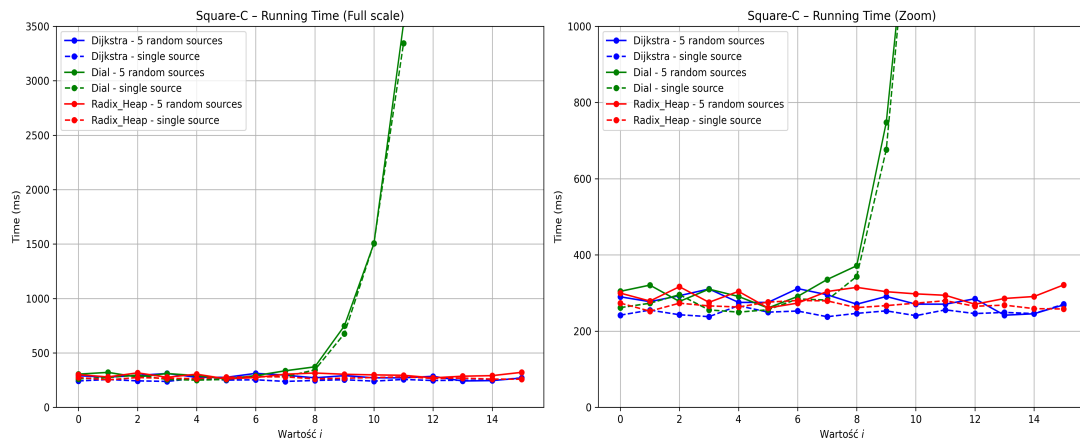
Rysunek 1: Zależność czasu od i dla rodziny Long-C

3.2 Long n



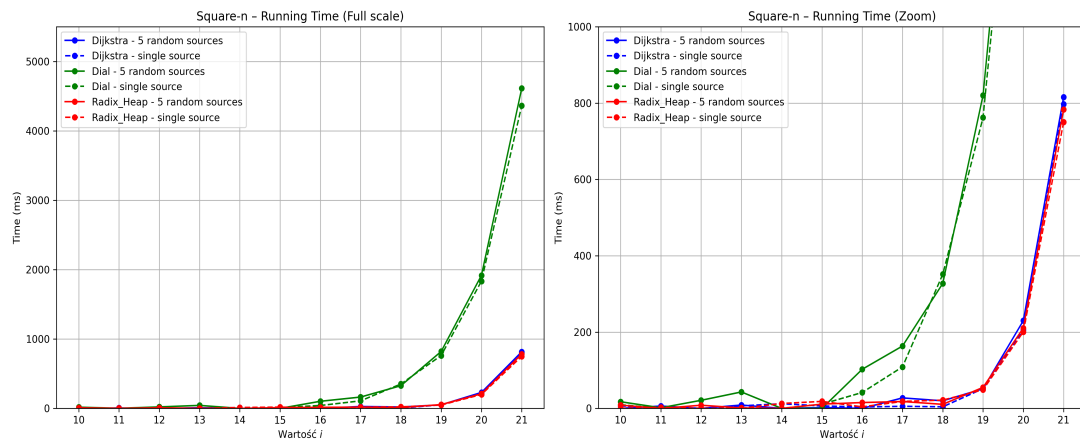
Rysunek 2: Zależność czasu od i dla rodziny Long n

3.3 Rodzina Square C



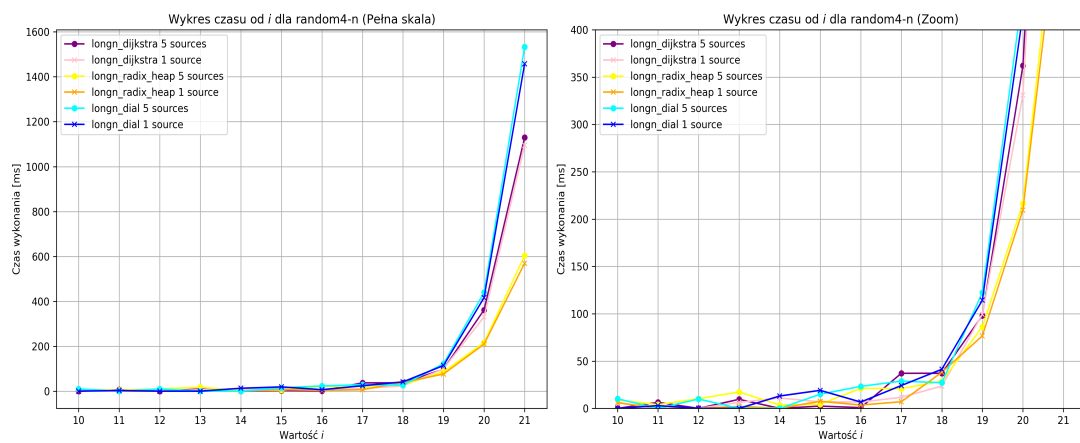
Rysunek 3: Zależność czasu od i dla rodziny Square C

3.4 Rodzina Square n



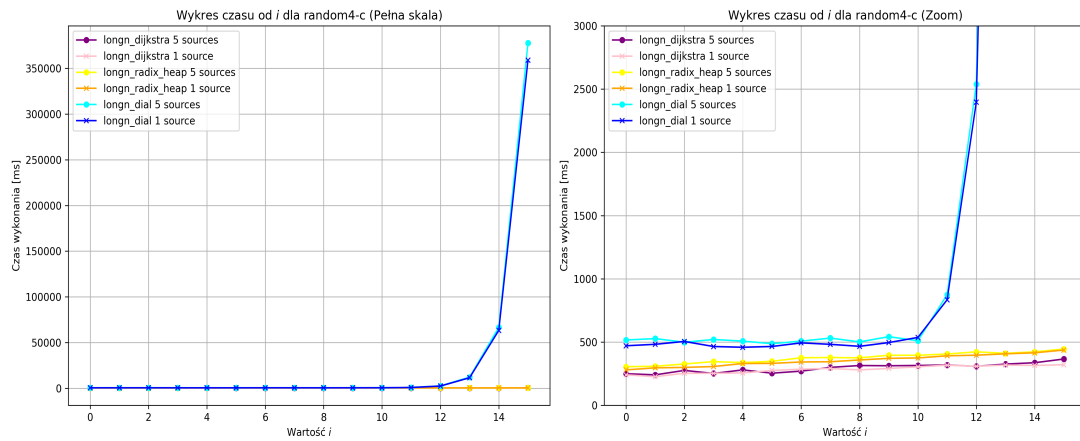
Rysunek 4: Zależność czasu od i dla rodziny Square n

3.5 random 4n



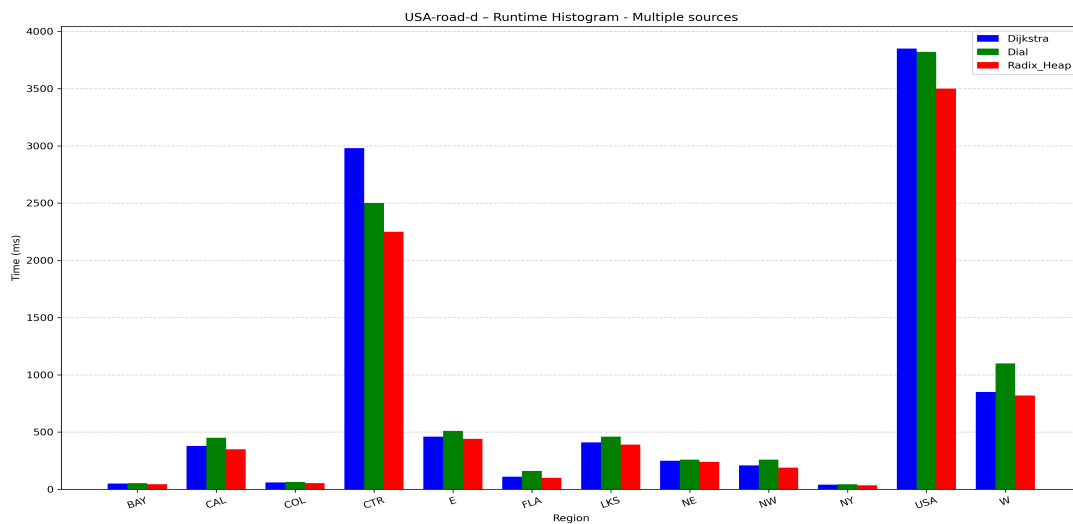
Rysunek 5: Zależność czasu od i dla rodziny random 4n

3.6 Rodzina random 4C

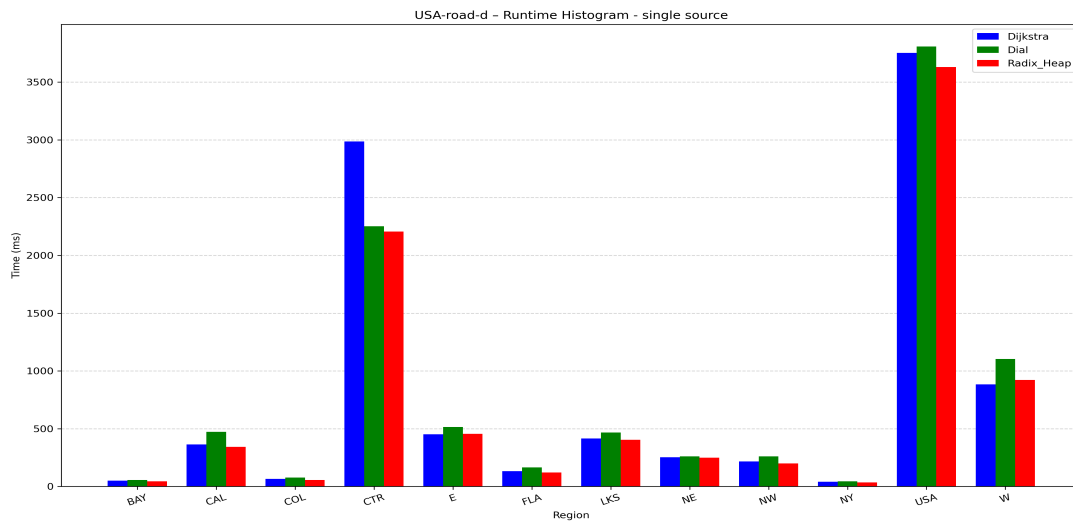


Rysunek 6: Zależność czasu od i dla rodziny random 4C

4 Wykresy słupkowe dla sieci drogowej USA-road-d



Rysunek 7: Histogram czasu działania dla grafów USA Road dla 5 punktów początkowych, uśrednione



Rysunek 8: Histogram czasu działania dla grafów USA Road 1 punktu początkowego

5 Wnioski i interpretacja

Algorytm Dijkstry radzi sobie solidnie dla wszystkich rodzin. Dla rodzin znacząco zwiększających C -maksymalny koszt krawędzi nie widzimy różnicy w zachowaniu algorytmu Dijkstry co jest zgodne z oczekiwaniami. Dla tych samych rodzin algorytm Diala zaczyna wykonywać się znacząco dłużej, co jest zgodne z oczekiwaniami, ponieważ jego złożoność jest silnie zależna od wartości C . Algorytm Radix Heap radzi sobie znacząco lepiej od algorytmu Diala, dając wyniki nieznacznie gorsze od algorytmu Dijkstry. Dla rodzin Square n , random $4n$ i long n algorytm Diala radzi sobie zdecydowanie lepiej, choć dalej gorzej od Dijkstry i RadixHeap. wraz ze wzrostem n widzimy oczekiwany wzrost czasu wykonywania algorytmów.

6 Tabela wyników dla największych instancji eskperymenty P2P

Poniżej przedstawione są wyniki eksperymentów P2P:

1. Od wierzchołku o najniższym indeksie do tego o najwyższym
2. Od losowych 4 wierzchołków (uśredniona wartość)

Tabela 1: Wyniki eksperymentów P2P

Rodzina	Algorytm	Start	Koniec	Odległość	Czas (ms)
Long-C	dijkstra	1	1048576	1308259008765	22.1503
Long-C	dial	–	–	–	brak (timeout)
Long-C	radix_heap	1	1048576	1308259008765	20.9841
Long-n	dijkstra	1	2097152	31336751771	281.450
Long-n	dial	1	2097152	31336751771	3512.10
Long-n	radix_heap	1	2097152	31336751771	249.215
Random4-C	dijkstra	1	1048576	3471241820	129.155
Random4-C	dial	1	1048576	3471241820	672.840
Random4-C	radix_heap	1	1048576	3471241820	94.8821
Random4-n	radix_heap	1	2097152	9051281	308.120
Random4-n	dial	1	2097152	9051281	1425.55
Random4-n	dijkstra	1	2097152	9051281	502.330
Square-C	dijkstra	1	1048576	122219500320	119.850
Square-C	dial	–	–	–	brak (timeout)
Square-C	radix_heap	1	1048576	122219500320	84.1102
Square-n	dijkstra	1	2096704	714640488	262.140
Square-n	dial	1	2096704	714640488	1021.50
Square-n	radix_heap	1	2096704	714640488	190.455
USA-road-d	dijkstra	1	23947347	23228284	3190.12
USA-road-d	dial	1	23947347	23228284	4788.60
USA-road-d	radix_heap	1	23947347	23228284	2315.95

Tabela 2: średnie czasy wykonywania

Rodzina	Algorytm	Czas (ms)
Long-C	dijkstra	119.421
Long-C	dial	brak (timeout)
Long-C	radix_heap	83.1550
Long-n	dijkstra	103.204
Long-n	dial	2398.50
Long-n	radix_heap	142.050
Random4-C	dijkstra	251.102
Random4-C	dial	633.901
Random4-C	radix_heap	210.880
Random4-n	dijkstra	298.410
Random4-n	dial	1433.10
Random4-n	radix_heap	361.225
Square-C	dijkstra	196.050
Square-C	dial	brak (timeout)
Square-C	radix_heap	84.9921
Square-n	dijkstra	440.115
Square-n	dial	1119.80
Square-n	radix_heap	336.502
USA-road-d	dijkstra	62.1042
USA-road-d	dial	4490.35
USA-road-d	radix_heap	42.8850

6.1 Wnioski P2P

Algorytmy Dijkstry i RadixHeap radzą sobie porównywalnie, z kolei Dial tak jak poprzednio słabo radzi sobie z rodzinami gdzie C jest duże, zgodnie z oczekiwaniami. Zgodnie z teoretycznymi oczekiwaniami Algorytm Diała nie powinien być wykorzystywany do danych gdzie C jest duże.